TinySQL - Text to Bigquery SQL generation

AAI-590-IN1 Capstone Project (Group 8)
April 2026

Santa Das Karmakar Krishnasamy Sethuraman Sanjay Kumar

Introduction: TinySql - Text To BigQuery SQL Generation

Context/ Approach

Modern enterprises face a critical trade-off when using LLMs for SQL. Large proprietary models deliver accuracy but introduce prohibitive costs, latency, and most critically data sovereignty risks by sharing sensitive information schema. We bridge this "Small Model Gap" by specializing a 2B parameter model exclusively for the BigQuery dialect.

4-bit QLoRA (Quantized Low-Rank Adaptation) for high-fidelity fine-tuning on a less memory foot print for edge-tier deployment.
Flash Attention 2 mechanism to reduce memory overhead and accelerate training.

Goal

Build a production-ready, Text-to-SQL assistant for BigQuery, by achieving high execution accuracy on complex analytical queries

AI Models

Gemma2-2B

Dataset

gretelai/synthetic_text_to_sql (Hugging Face)

Group

AAI-590-IN1 Group 8 Final Team Project

Objective, Business Case and value Proposition

Objective

Develop a lightweight, highly accurate "Text-to-SQL" translation engine specifically optimized for Google BigQuery dialect

Business Context

The 'SQL Barrier' prevents non-technical stakeholders from directly querying relational databases, resulting in data engineering backlogs. By automating BigQuery SQL generation, we remove the technical friction, allowing stakeholders to extract insights instantly while maintaining strict data sovereignty.

Business Value

Democratized Data Access: Empowers business users to query data using natural language.

- Cost & Time Efficiency: Reduces the bottleneck on data teams and accelerates reporting.
- Optimized Execution: Produces BigQuery-native SQL, minimizing syntax errors and execution costs.

Design Considerations

Model

Chose Gemma-2-2b-it—a lightweight LLM—to balance performance with computational constraints and fast inference times.

Prompt Engineering

Leveraged the Gemma template with <think> blocks for Chain-of-Thought (CoT) reasoning to "plan" JOINs and aggregations

Strict SQL Extraction

Designed a regex-based parser to strip conversational filler and extract raw SQL, preventing execution errors.

Scalable GKE/GCP

LoRA Fine-Tuning

Modular Pipeline

5-Layer Preprocessing Pipeline

Output: JSON lines (.jsonl) pushed to Google Cloud Storage (GCS)

Transpilation	Converting generic SQL to BigQuery dialect via **sqlglot**.
Normalization	Enforcing BigQuery safety rules (e.g., **SAFE_CAST**).
Schema Pruning	Injecting "distractor" schemas to improve model precision.
Value Grounding	Providing sample row hints for categorical columns.
Augmentation	Varying user prompt styles to ensure model robustness.

Input source: gretelai/synthetic_text_to_sql dataset.

Implementation Considerations

BigQuery Dialect Idiosyncrasies

Addressed BigQuery-specific syntax requirements (e.g., dataset anchoring {DATASET_ID}. table_name, backticks, avoiding strict MySQL constraints).

Dry Run Validation

Implemented an actual BigQuery client connection to run dry_run=True. This ensures generated SQL validates against the actual live schema.

Automated DIDL Setup

The pipeline automatically creates and tears down BigQuery tables based on the context DDL to validate SQL against a live environment

Training-Time Memory & Compute Optimizations

		Infrastructure & I/O Optimizations
4 bit Quantization (QLORA)	Utilized bitsandbytes (NF4, bfloat16) to drastically reduce memory footprint while maintaining model accuracy.	PyTorch Memory Management
PEFT/LoRA	Targeted attention mechanisms with r=16 to efficiently fine-tune the 2B parameter model without updating all base weights.	GKE GCS Fuse CSI
Training Hyperparameters	Paged AdamW 32-bit optimizer, cosine learning rate schedule, and gradient accumulation for stable convergence.	float16 (bf16) Precision

CI/CD Architecture

Containerization

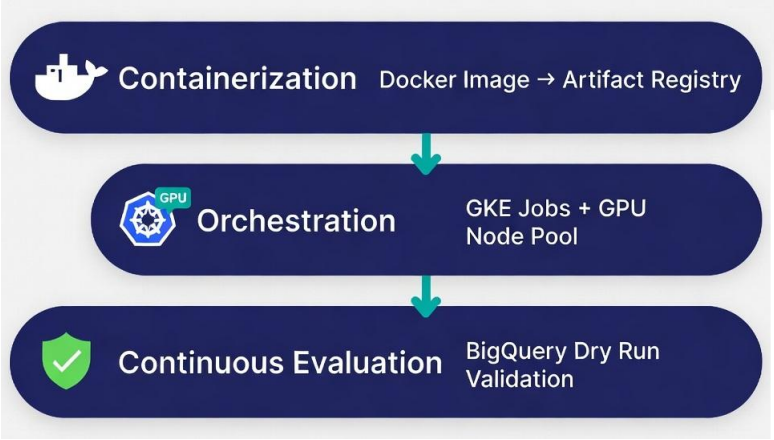
The inference and training environments are containerized. Docker images are built and pushed to Google Artifact Registry (gcloud builds submit).

Orchestration

Kubernetes (GKE) is used to deploy training and benchmarking jobs (e.g., gemma-vanilla_perf.yaml).

Continuous Evaluation

BigQuery Dry Run acts as an automated validation step within the pipeline, ensuring only syntactically sound SQL passes the benchmark



MLOps & Lineage

Data Lineage

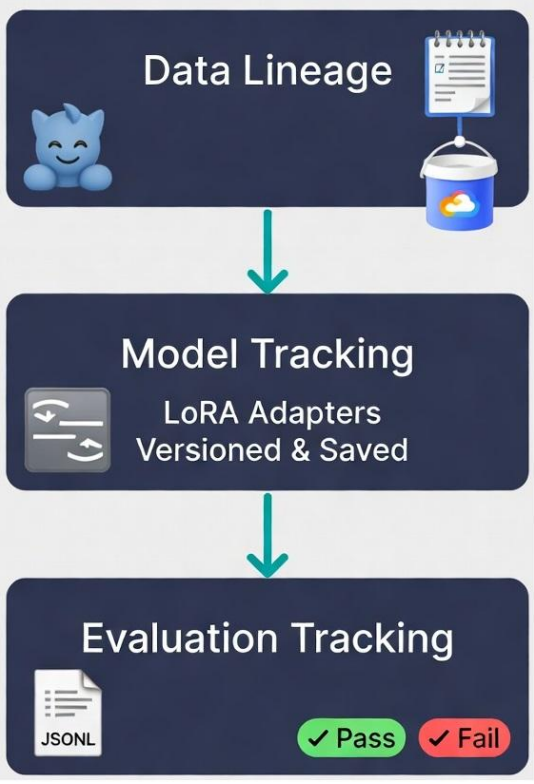
Data flows from **Hugging Face** (gretelai/synthetic_text_to_sql) → local preprocessing & augmentation → Google Cloud Storage

Model Tracking

Fine-tuned **LoRA adapters + tokenizers** are versioned and saved locally or in Google Cloud Storage for reproducible deployments and rollbacks

Evaluation Tracking

Inference results, generated SQL, and **BigQuery Dry Run** status (Pass/Fail) logged into evaluation JSON files

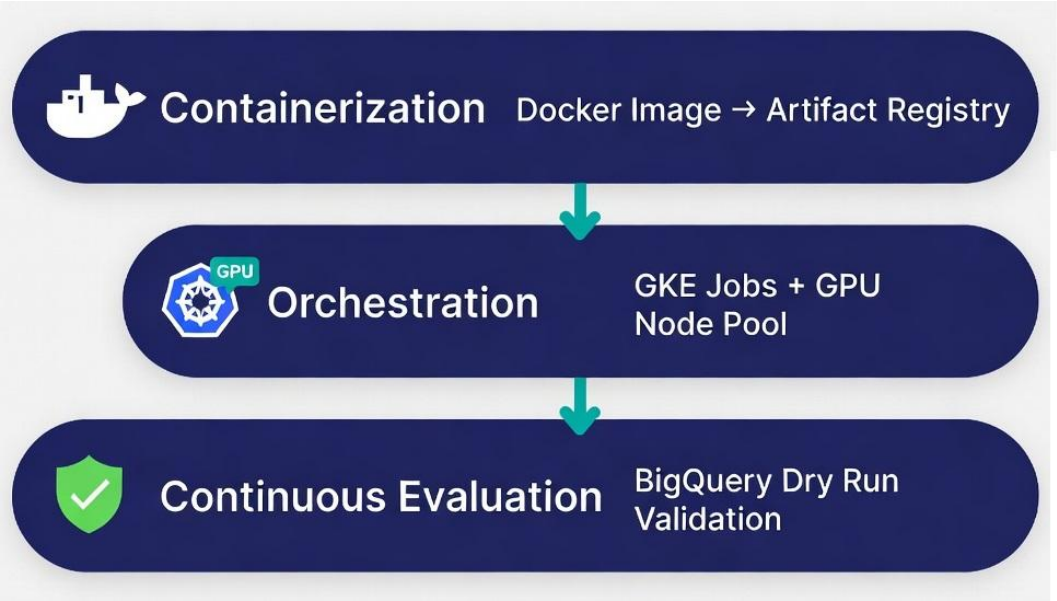


Optimization Results

Before vs After Optimization Table

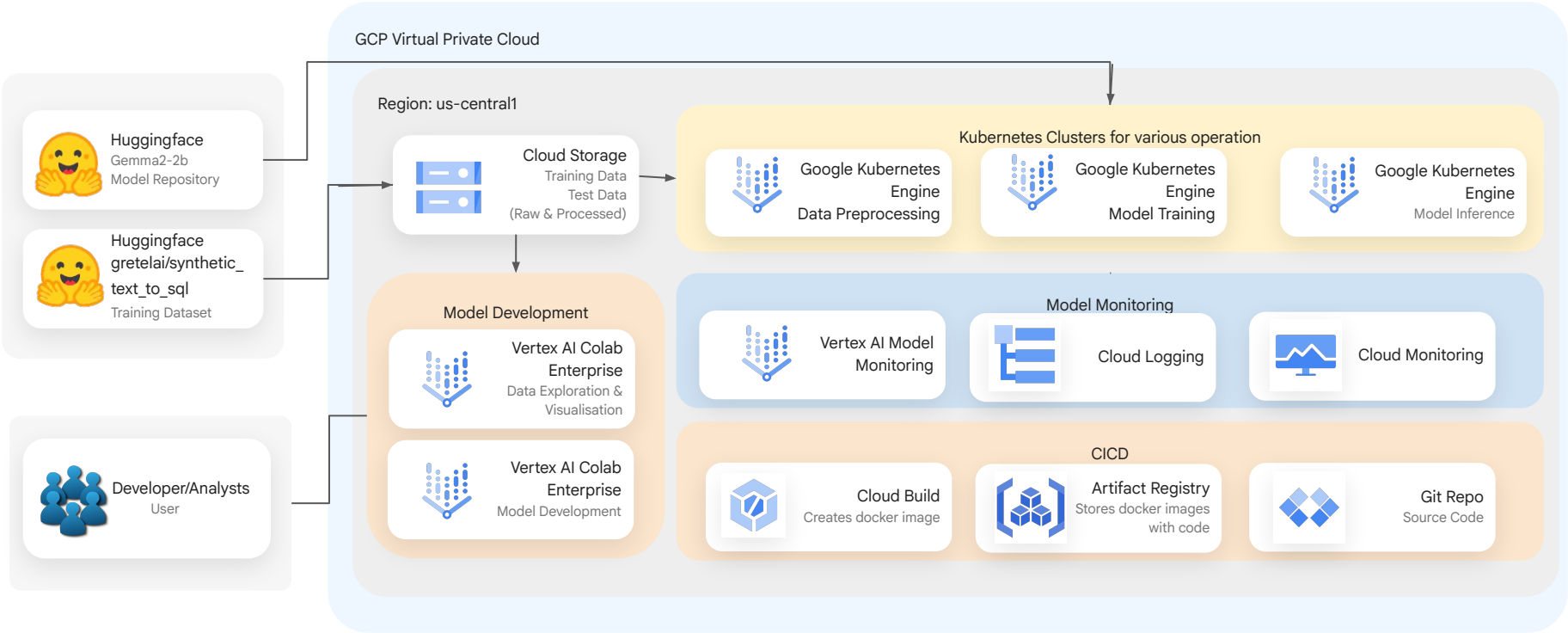
Aspect	Before Optimization	After Optimization (QLoRA + LoRA)	Improvement
Memory Usage	Very high (~60–80+ GB)	Significantly reduced (~20–25 GB)	↓ 60–70% memory footprint
GPU Requirement	Multiple high-end GPUs required	**Single NVIDIA L4 GPU**	1 GPU instead of cluster
Parameters Updated	100% of 2B model weights	**<1%** (only LoRA adapters)	99% fewer parameters trained
Training Speed	Slow due to full fine-tuning	Much faster with gradient accumulation & bf16	2–3x faster training
Hardware Cost	Very expensive (multi-GPU setup)	Low cost (single L4 GPU on GKE)	Major cost reduction
Precision	Usually full 32-bit	**bfloat16** mixed precision	Better efficiency & stability
Model Accuracy	Good but hardware-limited	Maintained or slightly improved	No loss in quality

CI/CD pipeline

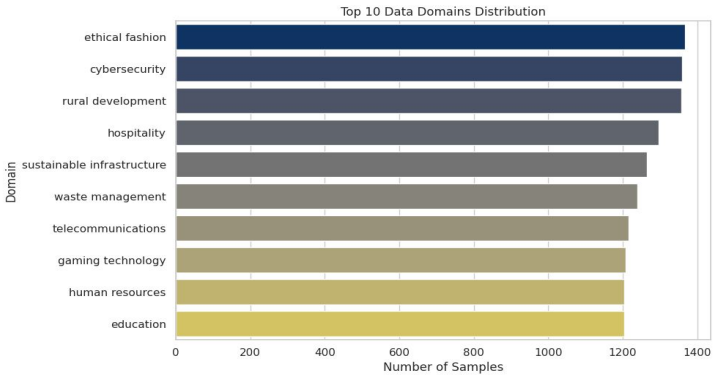


MLOps & Lineage

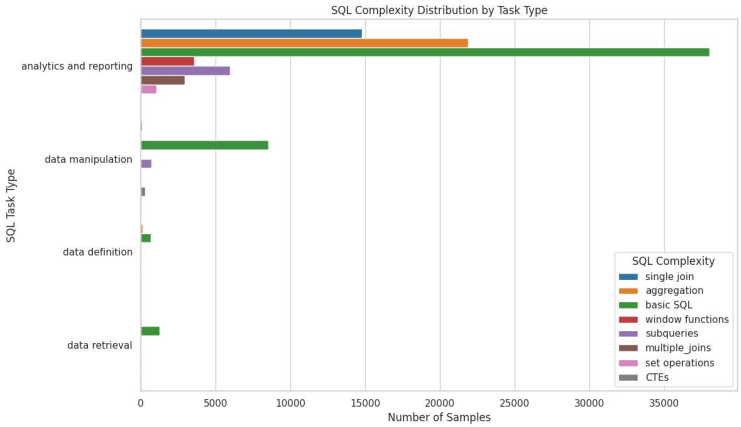
Overall Solution Architecture



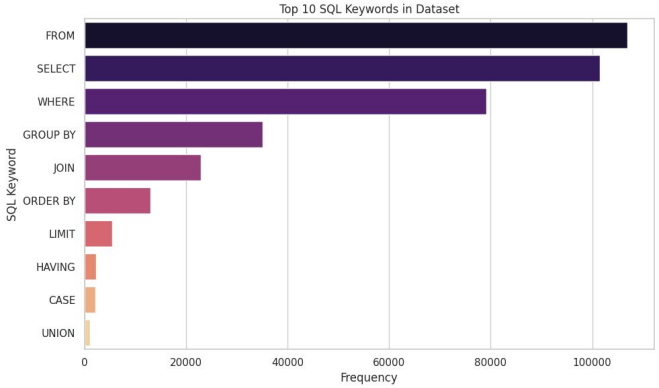
Training Data Diversity & Prompt Engineering Result



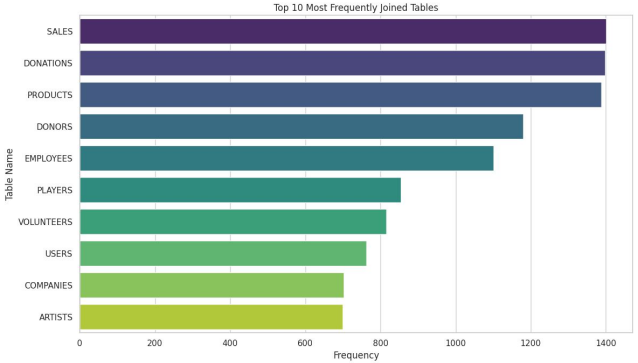
Equal representations from multiple domain



Most complex query patterns from analytic and reporting

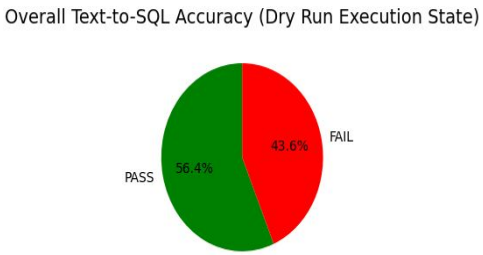


Complex query patterns with Union, Having etc

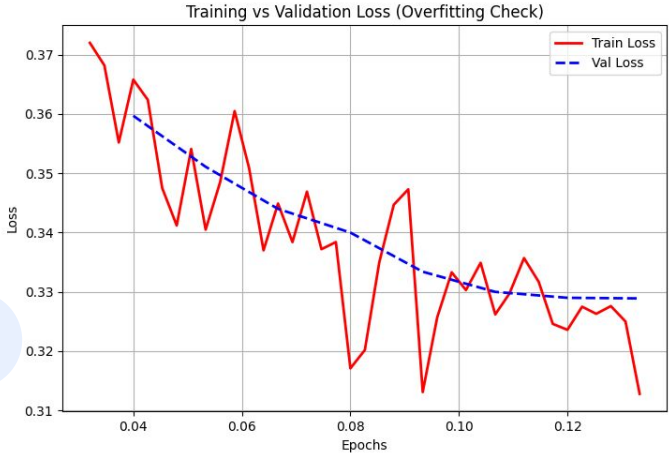


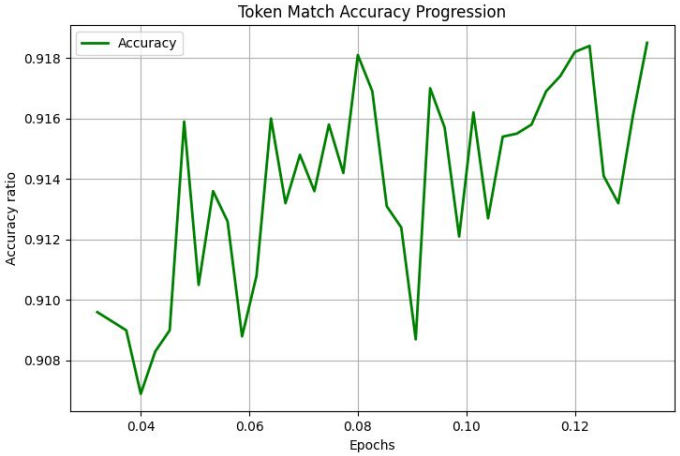
Mostly used tables in Joined queries

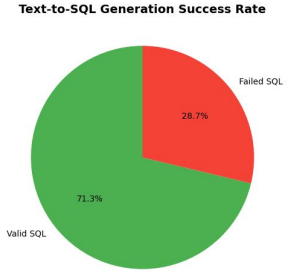
SFT Training Performance



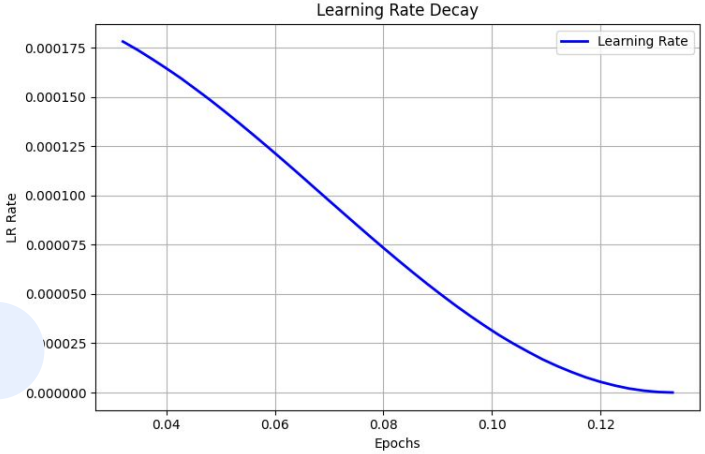
Successful Biquery validated query generated using **Prompt Engineering**

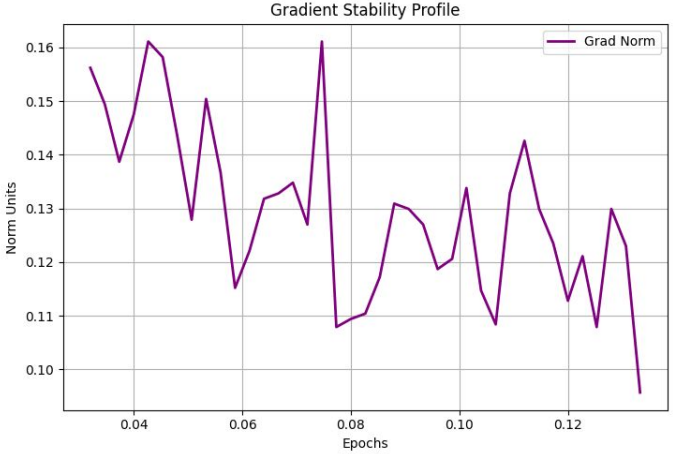






Successful Biquery validated query generated using **Prompt Engineering**





Future Enhancements

RAG Integration

Implement Retrieval-Augmented Generation to dynamically fetch relevant table schemas and DDLs for massive enterprise databases.

Self Healing SQL

Use error messages from BigQuery Dry Runs (e.g., "Unrecognized name") to prompt the LLM to fix mistakes in a multi-turn agentic loop.

Expanded Dialect Support

Leverage sqlglot to expand beyond BigQuery, seamlessly supporting Snowflake, Redshift, or PostgreSQL environments.

Conclusion

- TinySQL translates technical innovation into tangible business value.
- Democratizes data access, accelerates the time-to-insight for business users
- Reduces operational costs by producing optimized, native BigQuery queries

Colab Notebook

Thank you!